# Guía paso a paso - Práctico del 2do Parcial

Ingeniería y Calidad de Software - UTN FRC

## PARTE A - CAJA NEGRA: clases de equivalencia + valores límite + casos de prueba

> **Secuencia que tenés que seguir siempre:**
> **User Story -> criterios de aceptación -> condiciones externas -> clases válidas/inválidas -> valores límite -> casos de prueba -> precondiciones -> pasos -> resultado esperado.**

### 1. Leer la User Story y los criterios

Subrayá cada dato, estado o regla que pueda cambiar el comportamiento del sistema. No inventes restricciones. Si el requisito no dice que algo es obligatorio, numérico, mayor que cero, etc., no lo supongas.

### 2. Sacar las condiciones externas

Una condición externa es una entrada, estado o situación observable que afecta el resultado. Ejemplos: usuario/perfil, nombre, fecha, monto, formato de archivo, tamaño, cantidad, estado previo, existencia de otro registro.

### 3. Armar las clases de equivalencia

Para cada condición, separá grupos de valores que el sistema debería tratar de la misma manera. Ejemplo: rango válido 1..10 -> válida 1..10; inválida <1; inválida >10. Un dato concreto como 5 es un representante, no una clase.

### 4. Separar válidas e inválidas

Válidas: cumplen la regla. Inválidas: violan una regla explícita. Si hay varias formas distintas de violar la regla, pueden ser clases inválidas diferentes.

### 5. Revisar salidas

Si la solución de la cátedra pide clases de salida, agregá éxito y errores observables: mensaje de confirmación, rechazo, notificación, email, etc. No inventes mensajes que el requisito no define.

### 6. Aplicar Análisis de Valores Límite (AVL)

Usalo solo cuando existe una frontera ordenada: máximo, mínimo, rango, fecha límite, cantidad. Regla práctica: valor inmediatamente anterior, límite y valor inmediatamente posterior. Ejemplo máximo 500 MB -> 499 MB, 500 MB, 501 MB. No se usa AVL en categorías como PDF/Excel/Pantalla.

### 7. Elegir los casos mínimos

**Válidas se combinan.** Intentá cubrir muchas clases válidas en un mismo caso exitoso. **Inválidas se aíslan.** En un caso fallido, dejá todo válido salvo la clase inválida que querés comprobar.

### 8. Escribir precondiciones

Poné todo lo que debe existir ANTES del paso 1: usuario y permisos, datos cargados, relaciones, stock/saldo, fecha actual, estado de una máquina, registros previos, archivos disponibles, etc. Si el resultado depende de ese dato previo, declaralo.

### 9. Escribir pasos

Son las acciones del actor durante la prueba. Usá valores concretos: ingresar 'PUMA', adjuntar 'logo.png', seleccionar ranking 1, confirmar.

### 10. Escribir resultado esperado

Debe ser verificable y concreto. No escribas 'funciona bien'. Escribí qué muestra, registra, rechaza, calcula, envía o cambia el sistema.

### Reglas rápidas para no equivocarte

| Situación | Qué hacer |
|---|---|

| | |
|---|---|
| Muchas clases válidas | Combinarlas en un caso exitoso |
| Una clase inválida | Probarla sola; resto válido |
| Campo categórico | Partición de equivalencias; no AVL |
| Campo con límite/rango | PE + AVL |
| Dato no mencionado por requisito | No inventar restricción |
| Algo debe existir antes del test | Precondición |
| El usuario lo hace durante el test | Paso |
| Respuesta del sistema | Resultado esperado |

# PARTE B - KANBAN: cómo resolver el práctico

**Secuencia:**
**Contexto -> proceso real -> tablero -> WIP -> pull -> políticas -> clases de servicio -> bloqueos/cuello de botella -> métricas -> mejora.**

## 1. Identificar el flujo real

Leé el caso y buscá las etapas reales. No dibujes To Do / Doing / Done si el caso describe Análisis, Desarrollo, Testing, Cumplimiento, Homologación, etc.

## 2. Dibujar las columnas

Ponelas de izquierda a derecha según el proceso. Backlog al inicio y Done/Producción al final.

## 3. Colocar límites WIP

Poné un máximo de trabajo por etapa. El objetivo es limitar trabajo iniciado y evitar sobrecarga. No hay una fórmula universal: elegí un valor razonable según capacidad y contexto, y justificá que debe ajustarse empíricamente.

## 4. Aplicar sistema Pull

La etapa siguiente TRAE trabajo de la anterior cuando libera capacidad. Si Testing tiene WIP 2/2, Desarrollo no empuja un tercer ítem. Cuando Testing termina uno, recién entonces hace pull del siguiente.

## 5. Escribir políticas explícitas

Ejemplos: criterio para pasar de Análisis a Desarrollo, definición de Done, reglas para bloqueados, aprobación obligatoria, validación de cumplimiento, criterios para seleccionar nuevo trabajo y límites WIP.

## 6. Marcar clases de servicio

Si el caso lo permite: Expedite/Urgente, Fecha Fija, Estándar e Intangible. Expedite suele tener tratamiento especial y WIP muy limitado; Fecha Fija se gestiona según una fecha comprometida.

## 7. Marcar bloqueados

Un ítem bloqueado sigue siendo WIP. Hacelo visible en el tablero.

## 8. Detectar cuello de botella

Es la etapa que limita el flujo y acumula trabajo delante. No lo 'resuelvas' simplemente aumentando WIP; gestioná capacidad, políticas, dependencias o prioridades.

## 9. Agregar métricas si lo piden

Lead Time = solicitud a entrega. Cycle Time = inicio del trabajo a fin. Touch Time = tiempo realmente trabajado. Throughput = ítems terminados por unidad de tiempo.

## 10. Explicar mejora continua

El tablero y las métricas sirven para experimentar: cambiar WIP, políticas o asignación de capacidad y verificar si mejora el flujo.

## Plantilla rápida para dibujar

**Backlog -> Análisis [WIP] -> Desarrollo [WIP] -> Testing [WIP] -> Aprobación/Homologación [WIP] -> Done**

Debajo o al costado: políticas explícitas. Si corresponde, agregá carriles horizontales para Expedite / Fecha Fija / Estándar / Intangible.

# PARTE C - PLANTILLA DE RESPUESTA PARA EL EXAMEN

## Si te piden clases de equivalencia

1) Condición externa.
2) Clases válidas.
3) Clases inválidas.
4) Repetir para todas las reglas de la US.
5) Agregar clases de salida si el template las incluye.

## Si te piden un caso exitoso

**Nombre:** Registro/operación exitosa...
**Prioridad:** Alta/Media/Baja según impacto.
**Clases cubiertas:** listar las válidas que combinaste.
**Precondiciones:** actor, permisos, datos, relaciones, estado inicial.
**Pasos:** acciones con datos concretos.
**Resultado:** éxito observable y exacto.

## Si te piden un caso fallido

Elegí UNA clase inválida principal. Todo lo demás debe quedar válido. El resultado esperado debe ser el rechazo o mensaje correspondiente, y el sistema no debe completar la operación.

## Si te piden maximizar clases

Combiná en un único caso todas las clases válidas compatibles entre sí y aprovechá valores límite válidos. Ejemplo: si el máximo permitido es 500 MB, usar exactamente 500 MB permite cubrir la clase válida y el límite.

## Chequeo final de 30 segundos

• ¿Usé solo reglas que aparecen en la User Story/criterios?

• ¿Identifiqué todas las condiciones externas?

• ¿Separé correctamente válidas e inválidas?

• ¿Apliqué AVL solo donde hay límites?

• ¿Combiné válidas y aislé inválidas?

• ¿Las precondiciones describen el estado previo?

• ¿Los pasos son acciones concretas?

• ¿El resultado esperado es verificable?

• En Kanban: ¿hay WIP, pull, políticas, clases de servicio/bloqueos si corresponde?